プログラミング言語実験 -第 2 回-

2412747 江尾祐哉

1 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai2-1.cpp -o ./2412747_kadai2-1」とコンパイルし、
「./2412747_kadai2-1 x y」のようにして、x と y それぞれに数値を入力して実行する。

・実装概要

最大公約数を求める関数を作成し、そこに与えられた正の整数を入力することで、求めるべき数値を出力するというのがプログラムの全体像である。そのためには、まず入力された値についての吟味をしなくてはならなく、

1. 引数の数が 2 つではない
2. 引数にそもそも数字ではない英語や記号などが含まれている
3. 数字が少数や負の数である

という 3 点についてエラーの処理を行う。しかし、今回は 2 と 3 を共通にして、入力した文字列に対して全ての文字が'0'～'9'の間にあるかを調べており、エラーメッセージも「引数に 0～9 の数字以外の文字が入っています。正の整数を入力してください。」というようにまとめてある。これは 2 の段階で 3 の処理もまとめて行えるからだと考え、またこれを細分化したとしても冗長なコードが生成されてしまうため、無意味であると感じたからだ。

このエラー処理が終了したら、文字列をそれぞれ x と y に atoi 関数を使って代入していく。課題文の指示ではこれらの数値を m と n に代入するよう言われているが、いきなり代入せずにわざわざ x と y に入れているのは、「まずは文字列を数値に変換します」という工程を明示するためであるのと、次に行う最大公約数の計算で m と n の大小関係に関わらず処理させるためである。最大公約数の計算は m を n で割る計算をするので、x と y の大小関係によって m と n のどちらに代入するかを場合わけした。こ

れで、入力値の大小がどちらでも良いように対策した。

最大公約数の計算は課題文で指示されたように、

1. m を n で割ったあまりを r に代入
2. r が 0 ならそのときの n が最大公約数
3. r が 0 でないならば n を m に、r を n にして再度計算

という流れを踏んだものになっている。ここで 0 除算が起こらないように calc_gcd 関数の処理に入る前に数値が 0 でないかを判別している。そもそも最大公約数の計算で 0 が含まれているのはおかしいからでもある。計算が終了したら、最終的にメッセージを出力して完了となる。

・実行結果

```
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-1 -1 5
引数に0~9の数字以外の文字が入っています。正の整数を入力してください。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-1 0 9
正の整数を入力してください。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-1 4 6
4と6の最大公約数は2です。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-1 1 9 10
引数の数が2つではありません。入力し直してください。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-1 1.3 5
引数に0~9の数字以外の文字が入っています。正の整数を入力してください。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-1 35 9865
35と9865の最大公約数は5です。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-1 60 1404
60と1404の最大公約数は12です。
```

図1　問題1の実行結果

2 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai2-2.cpp　-o ./2412747_kadai2-2」とコンパイルし、「./2412747_kadai2-2 array1_2026.txt array2_2026.txt」というように、引数に数列の記述してあるテキストファイル名を指定して実行する。

・実装概要

この課題では2つのファイルの両方に対応させるプログラムを作成しなければならず、1 つのファイルには整数の羅列が、もう 1 つのファイルには小数の羅列がされている。両方に対応させるために今回は関数テンプレートを使用するよう指示があったため、これを使用しての実装となっている。

数値の合計はごく単純で、授業資料のサンプルコードにあったものをそのまま流用している。すなわち、引数に 2 つの数値を入れるとその合計が返ってくるという関数テンプレートである。これを利用して while ループを回し、またループをするごとに fin1_len(もしくは fin2_len)という変数をインクリメントし、これにより行数も数えることができる仕組みになっている。もちろん int 型でも double 型でも対応するプログラムとなっており、コード全体の可読性も高いと言える。これより、数字の合計と行数を簡単に求めることができ、あとはこれらを出力すれば完了となる。

なお、宣言した calc_gcd 関数には static という修飾子がついているが、これによりソースファイル内のみに関数のスコープを適用させ、他のファイルに干渉しないようにさせている。今回はディレクトリ内に複数ファイルがあるといえ、コンパイルは 1 つずつ行う上、同じ関数を命名するようなことはないと言えるが、いずれ開発の規模が大きくなったときに、同じ名前の関数の宣言や Makefile によるコンパイルを行うことが考えられ、それによってエラーが発生する恐れがあると考えられるので、あらかじめ習慣づけておくという意味でも static をつけて関数の宣言をした。以後の課題でも同様に行なうよう意識している。

・実行結果

```
e2412747@a74:~/project/cpj2026_2412747_2$ g++ 2412747_kadai2-2.cpp -o 2412747_ka
dai2-2
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-2 array1_2026.txt arr
ay2_2026.txt
1つ目のファイルは361行で、記載されている数字の合計は178396です
2つ目のファイルは260行で、記載されている数字の合計は13028.2です
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-2 1 2 3
引数の数が2つではありません。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-2 1 2
1つ目のファイルを開けませんでした。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-2 array1_2026.txt no_
file
2つ目のファイルを開けませんでした。
```

図2　問題2の実行結果

3 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai2-3.cpp -o ./2412747_kadai2-3」とコンパイルし、「./2412747_kadai2-3 x y」というように、x と y それぞれに格子点の座標を入力して実行する。

・実装概要

この課題は再帰関数を使用したアルゴリズムの基本的な問題である。再帰関数とは、関数の中で自分自身の関数を呼び出して処理を繰り返して行なっていく関数のことである。再帰関数の実装を行う上で一番重要な点があり、それは終了条件を明記することである。終了条件を書かなかった場合、無限に関数の処理が行われプログラムが正常に実行されないので、この点にはしっかりと留意する必要がある。

今回は碁盤目状の格子路の経路計算であり、入力された座標を終点として始点(0, 0)からの経路数を計算することとなる。しかしながら終点は可変であるため、始点→終点という経路を計算するプログラムの実装は複雑になる。したがって、終点→始点という方向から経路を計算することで、実装を単純にしている。そのため経路計算の関数である calc_route の終了条件は x が 0 でかつ y が 0 のときとなっている。始点→終点の進み方は右もしくは上であるため、終点→始点の進み方は左もしくは下となる。また、今いる地点からの進み方は左もしくは下の 2 通りであるため、これより calc_route 関数の返り値は、calc_route(x - 1, y) + calc_route(x, y -1)となることがわかる。また、この問題では(4, 2)の座標が工事中となっている。そのため、この座標の時を例外処理として if 文で扱い、0 を返すようにしている。これにより経路計算の関数は完成となる。

プログラム全体としては、まず引数の数のエラー処理と、引数が正の整数以外であるのは不適当なためそれのエラー処理をしている。その後 atoi 関数を用いて int 型の変数に代入している。ここで(4,2)は工事中で終点として指定することができないので、(4, 2)の入力は工事中である旨を告げて弾くようにしている。その後、calc_route 関数の処理をし、最終的に計算結果が求まることとなる。

・実行結果

```
e2412747@a74:~/project/cpj2026_2412747_2$ g++ 2412747_kadai2-3.cpp -o 2412747_ka
dai2-3
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-3 7 5
終点座標が(7,5)のときの経路数は492です。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-3 1 1
終点座標が(1,1)のときの経路数は2です。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-3 0 0
終点座標が(0,0)のときの経路数は1です。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-3 -2 3
引数に0~9の数字以外の文字が入っています。正の整数を入力してください。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-3 1.3 6
引数に0~9の数字以外の文字が入っています。正の整数を入力してください。
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-3 4 2
(4,2)は工事中なため使用できません。
```

図3　問題3の実行結果

4 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai2-4.cpp -o ./2412747_kadai2-4」とコンパイルし、「./2412747_kadai2-4 strings2026/numbers_2026.txt」というように、引数に数字列もしくは文字列の記述されたテキストファイルを指定して実行する。

・実装概要

この課題はファイルからの適切な読み込みとソートアルゴリズムの実装の 2 つの目標達成が必要とされる。

まずはファイルからの適切な読み込みについてだが、今回は数字列のみが記述されたファイルと文字列のみが記述されたファイルの 2 種類に対応しなくてはならない。もちろん 2 つのファイルの内容は違う上、行数も異なる。そのためプログラム内で静的に配列を確保する方法は両者に柔軟に対応するという指示を満たさない。また、各行の文字数が違うことや数字列の記されたファイルには数万行もの数の数字列が記述されてあることから、new を利用した動的な 2 次元配列の使用も好ましくないと考えられる。そこで調べてみたところ、C++独自のものとして vector コンテナというものが存在した。Vector は各要素を線形に順序を保ったまま格納していくコンテナで、配列とは異なりストレージを vector そのものが管理するために、領域の拡張が自動的に行われるものである。また、豊富なメンバ関数により要素のアクセスや挿入および削除が容易である

ことが特徴である。これを利用することで、ファイルの読み取りを動的に行う上に可読性の高いコードが実現できることになると考えた。

エラー処理やファイルの読み込みが終わったら、まずは string 型の vector コンテナ、すなわち可変長の文字列型の配列の file_content を定義した。これにファイルから読み取った文字列を代入していくが、文字列の挿入には vector コンテナのメンバ関数である push_back を使用することで安易なものとなる。配列の大きさもメンバ関数の size を使用して取得した。次に string の要素が文字列であるか、数字列であるかによって処理を変えなくてはならないので、if 文によって string の一番始めの要素を参照し、それが数字か英語大文字かで条件分岐させた。これは今回のテキストファイルの形式であるからできる処理であり、汎用性はそこまでない。数字列であるなら、今度は int 型の vector コンテナである file_nums を定義し、そこに string 型を int 型に変換させる stoi 関数を使用しながら、メンバ関数の push_back で挿入していった。そして file_descending_sort 関数に引数として file_nums を代入した。文字列である場合はそのまま file_descending_sort 関数に代入した。今回は課題文の指示に関数のオーバーロード機能を使用するよう明記されていたので、file_descending_sort 関数でそれを利用した。

降順ソートのアルゴリズムであるが、今回は一番単純なバブルソートを用いた。これは配列があったとき、ある要素とその次の要素を比較して降順でなければ値を交換、そして次の要素とその次の要素を比較、そして・・・として最後までソートし、1周が完了したら次に始点を 1 つ隣にずらし、また比較・・・、そして最後まで比較し終えたら要素が降順にソートされているというものである。選択理由としては実装が容易で、コードも単純であるため理解しやすいことが挙げられるが、一方でオーダーが$O(n^2)$であるのが懸念点であると考えられる。file_descending_sort 関数の内部では引数が文字列でも数字列でも同様にこの処理をしている。なお要素と次の要素を比較する関係上、要素数全体を len としたときに、for 文の条件を i < len　ではなく、i < len - 1 としなくてはいけないことに注意した。また、引数にアドレスを指定することで、引数の配列そのものをいじる仕様にしたため、関数の型は void 型となっている。

これで最後に要素全てを出力する処理を行えば、ファイル内の文字列もしくは数字列が降順で出力されることとなる。要素一つ一つはカンマで区切り、また最後の要素の出力の後に改行を出力するようにした。あとはこの問題と次の 5 番では、ファイルのオープンに失敗したときに、cerr でベタ打ちするのではなく、errno を利用した perror 関数によってエラーメッセージを出力するようにした。ファイルのオープンに失敗すると

いうのは、ユーザーの利用によるものではなくコンピュータの方に委ねられるものであるため、errno を利用し、perror 関数によって自身が記述したエラーメッセージとシステムのエラーメッセージを両方出力させる方が親切(ファイルのオープンに失敗した原因の特定が容易)であると考えたからである。しかし C 言語であれば、C 言語というものは OS のシステムコールと密であるため、この利用方法は妥当だと考えられるが、C++である場合にはこれが果たして正しいのかどうかというのは定かではないので、一考の余地があると感じている。

・実行結果

数字列に関しては量が膨大なため、一部抜粋とした。

```
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-4 strings2026/numbers_2026.txt
10000, 10000, 10000, 10000, 10000, 9998, 9998, 9998, 9998, 9998, 9998, 9997, 9997, 9997, 9997,
 9997, 9997, 9997, 9997, 9996, 9996, 9996, 9996, 9996, 9996, 9995, 9995, 9995, 9995, 9995, 9994, 999
4, 9994, 9994, 9994, 9994, 9994, 9994, 9994, 9993, 9993, 9993, 9993, 9993, 9993, 9993, 9993, 9992, 9
992, 9992, 9992, 9992, 9992, 9992, 9991, 9991, 9990, 9990, 9990, 9989, 9989, 9989, 9989, 9989, 9989,
 9989, 9989, 9988, 9988, 9988, 9988, 9987, 9987, 9987, 9987, 9986, 9986, 9986, 9985, 9985, 9985, 998
4, 9984, 9984, 9984, 9983, 9983, 9983, 9983, 9983, 9983, 9982, 9982, 9982, 9982, 9982, 9982, 9981, 9
981, 9981, 9980, 9980, 9980, 9979, 9979, 9979, 9979, 9979, 9979, 9978, 9978, 9978, 9978, 9978, 9978,
 9977, 9977, 9977, 9977, 9977, 9977, 9976, 9976, 9976, 9976, 9976, 9976, 9975, 9975, 9975, 9975, 997
5, 9974, 9974, 9974, 9974, 9974, 9974, 9974, 9973, 9973, 9973, 9973, 9973, 9972, 9972, 9972, 9972, 9
972, 9972, 9972, 9971, 9971, 9971, 9971, 9971, 9971, 9971, 9971, 9971, 9970, 9970, 9970, 9970, 9970,
 9970, 9970, 9970, 9970, 9970, 9970, 9970, 9969, 9969, 9969, 9969, 9969, 9968, 9968, 9968, 9968, 996
7, 9967, 9967, 9966, 9966, 9966, 9966, 9966, 9965, 9965, 9965, 9965, 9964, 9964, 9964, 9963, 9963, 9
963, 9963, 9963, 9963, 9963, 9963, 9963, 9963, 9963, 9963, 9963, 9962, 9962, 9962, 9962, 9962, 9961,
 9961, 9961, 9961, 9961, 9961, 9961, 9961, 9961, 9961, 9960, 9960, 9960, 9960, 9960, 9960, 9960, 995
9, 9959, 9958, 9958, 9958, 9958, 9958, 9958, 9958, 9958, 9957, 9957, 9957, 9957, 9957, 9957, 9956, 9
956, 9956, 9956, 9955, 9955, 9955, 9954, 9954, 9954, 9954, 9954, 9954, 9954, 9954, 9954, 9954, 9954,
 9954, 9953, 9953, 9953, 9953, 9952, 9952, 9952, 9952, 9952, 9951, 9951, 9950, 9950, 9950, 9950, 995
0, 9950, 9949, 9949, 9948, 9948, 9948, 9948, 9948, 9948, 9948, 9948, 9947, 9947, 9947, 9947, 9947, 9
946, 9946, 9946, 9946, 9946, 9945, 9945, 9945, 9945, 9945, 9945, 9944, 9944, 9944, 9944, 9944, 9944,
 9943, 9943, 9943, 9943, 9943, 9943, 9943, 9942, 9942, 9942, 9942, 9942, 9942, 9942, 9942, 9942, 994
2, 9941, 9941, 9941, 9940, 9940, 9940, 9940, 9940, 9940, 9940, 9940, 9939, 9939, 9939, 9939, 9938, 9
938, 9938, 9938, 9938, 9937, 9937, 9937, 9937, 9937, 9937, 9936, 9936, 9936, 9936, 9936, 9935, 9935,
 9934, 9934, 9934, 9934, 9933, 9933, 9933, 9933, 9933, 9932, 9932, 9932, 9932, 9932, 9932, 9932, 993
1, 9931, 9931, 9930, 9930, 9930, 9930, 9930, 9930, 9930, 9930, 9929, 9929, 9929, 9929, 9928, 9928, 9
928, 9928, 9928, 9928, 9927, 9927, 9927, 9926, 9926, 9926, 9926, 9926, 9925, 9925, 9925, 9925, 9925,
 9925, 9925, 9925, 9924, 9924, 9924, 9924, 9923, 9923, 9922, 9922, 9922, 9922, 9922, 9922, 9922, 992
2, 9922, 9922, 9921, 9921, 9921, 9921, 9921, 9921, 9921, 9920, 9920, 9920, 9920, 9919, 9919, 9919, 9
918, 9918, 9918, 9918, 9918, 9918, 9917, 9917, 9917, 9917, 9917, 9916, 9916, 9916, 9916, 9915, 9915,
 9915, 9914, 9914, 9914, 9914, 9914, 9913, 9913, 9913, 9913, 9913, 9913, 9913, 9912, 9912, 9912, 991
2, 9912, 9912, 9911, 9911, 9911, 9911, 9911, 9910, 9910, 9910, 9910, 9909, 9909, 9909, 9909, 9909, 9
909, 9908, 9908, 9908, 9907, 9907, 9906, 9906, 9906, 9906, 9906, 9906, 9906, 9905, 9905, 9905, 9905,
 9905, 9905, 9904, 9904, 9904, 9904, 9903, 9903, 9902, 9902, 9902, 9902, 9902, 9902, 9901, 9901, 990
1, 9901, 9901, 9901, 9901, 9901, 9901, 9900, 9900, 9900, 9899, 9899, 9899, 9899, 9898, 9898, 9898, 9
898, 9897, 9897, 9897, 9897, 9897, 9897, 9897, 9896, 9896, 9896, 9896, 9895, 9895, 9895, 9895, 9895,
 9895, 9895, 9894, 9894, 9894, 9894, 9893, 9893, 9893, 9893, 9893, 9892, 9892, 9892, 9892, 9892, 989
2, 9892, 9892, 9891, 9891, 9890, 9890, 9889, 9889, 9889, 9889, 9889, 9888, 9888, 9888, 9888, 9888, 9
887, 9886, 9886, 9886, 9886, 9886, 9886, 9885, 9885, 9885, 9885, 9885, 9884, 9884, 9884, 9884, 9884,
 9883, 9883, 9882, 9882, 9882, 9881, 9881, 9881, 9881, 9881, 9880, 9879, 9879, 9879, 9879, 9878, 987
8, 9878, 9878, 9877, 9877, 9877, 9876, 9876, 9876, 9876, 9876, 9875, 9875, 9875, 9875, 9875, 9875, 9
874, 9874, 9874, 9874, 9874, 9874, 9874, 9874, 9874, 9873, 9873, 9873, 9873, 9873, 9872, 9872, 9872, 9872, 9871,
 9871, 9870, 9870, 9870, 9870, 9870, 9870, 9870, 9869, 9869, 9869, 9869, 9869, 9869, 9869, 9868, 986
```

図4-1　問題4の実行結果(一部抜粋)

```
e2412747@a74:~/project/cpj2026_2412747_2$ ./2412747_kadai2-4 strings2026/strings_2026.txt
ZPOR, Z, YUZQ, Y, WDVB, VEYY, UNIG, TCXT, T, RTY, RTG, PRHZ, PEDJ, OXIS, OQXS, OEWE, NQPY, MNB, MMZG
, KL, KFUF, JZMX, JRGR, JQCW, IJCI, HJTD, HELL, GPKE, GLTE, GH, FSLF, FLTI, FBPD, DKPO, CBN, CANL, B
ZR, BXQP, BAZX, AS, AABC, AAAA, A
```

図4-2　問題4の実行結果

5 問目

・コンパイルおよび実行方法

「g++ 2412747_kadai2-3.cpp -o ./2412747_kadai2-5」とコンパイルし、「./2412747_kadai2-5 file2026/」というように、引数にバイナリファイルが存在するディレクトリのパス名を指定して実行する。

・実装概要

この課題は前 2 つのような簡易なアルゴリズムを実装できるかを問うようなものでは一切なく、コンピュータのシステムの処理について簡易的なものを実装することができるかを聞いているものであり、難易度が高いと感じた。その理由としてはシステムコールの処理を、簡単ではあるが行わないといけない点や、IED の C++のバージョンが C++14 であるため、C++17 で軒並み実装された便利な関数等が使用できない点が挙げられる。C 言語でシステムコールに関するプログラムを作成したことがあれば馴染み深いものだが、そうでない人にはかなり取っ付きにくい課題だと思った。

まず、このプログラムは引数にファイル名ではなくディレクトリのパス名を渡さなければならない。C++17 であれば filesystem というものがあるらしいが、今回は使用できないので dirent.h と呼ばれる C 言語のライブラリを使用してディレクトリの操作を行うライブラリ内の opendir 関数を使用してディレクトリを開き、DIR 構造体に情報を代入している。このとき opendir 関数は C 言語の関数であるため、引数も対応させるために string 型のメンバ関数である c_str を使用している。もちろんファイルの時と同様にディレクトリの open でエラーが起こりうるので、エラーの処理をしている。

次に読み取るファイル名を保存するための string 型の vector コンテナと、ディレクトリ操作を行うための dirent 構造体の宣言をする。この dirent 構造体には、エントリのファイル番号の d_fileno、レコードの長さの d_reclen、ファイルタイプの d_type、ファイル名の文字列長の d_namlen、ファイル名の d_name が含まれている。ディレクトリを読み取る readdir 関数のために、この構造体が必要となり、ループ処理でディレクトリ内のファイルを一つずつ読み取るコードとなっている。ループ内では、まず読み取ったエントリの名前を string 型の name に代入する。その後にこの name を filename に追加していくのだが、この処理を行う前に幾つかの手順を踏んでいる。

1. 「.」と「..」を除外するために if 分岐
2. 読み取ったファイル名をディレクトリのパス名と接続
3. 通常ファイルかどうかの判定

ディレクトリ内には「.」と「..」というものが含まれているが、これは自分の inode(ファイルのメタデータをまとめたもの)と親の inode を意味する。そのため、もしこれを除外しなかった場合、readdir 関数でこれらの inode を読み取って、延々と読み取りの操作を行いループが終わらないということが発生する。したがって 1 番の処理が必要となる。通常ファイルかどうかの判定を行う stat 関数ではどのファイルかを完全なパスで判断するので、カレントディレクトリがどこであってもプログラムを動作させるために 2 番の処理を行っている。また、今回はディレクトリ内に 10 個のバイナリファイルのみが含まれていることとなっているが、一般的なディレクトリにはこれらだけでなくサブディレクトリやシンボリックリンク、その他特殊ファイルなども含まれていることが想像できる。そのため、それらのファイルを filename に追加しないように 3 番の判定を行なっている。判定のための is_regular_file 関数では、ファイルに関する情報を保持する stat 構造体を用いて、stat システムコールでパスとして指定したファイルの情報を検索している。そしてそれが通常のファイルかどうかをテストする S_ISREG マクロによって真偽を返しているという仕組みになっている。そうして close することでディレクトリの操作は終了となる。

次にファイルのソートを行なっているが、これは後で説明することにする。ディレクトリ内にマルウェア A ファミリーがあったかどうかを保持する bool 型の is_finde を宣言して、その後 filename に入れたファイルを一つずつ調査していくこととなる。ここで、マルウェア A ファミリー内に見られる文字列はバイト列で示されている。このバイト列は const unsigned char の配列としてマクロを定義してあるが、バイト列の検索のためファイルもバイナリ形式で読み取らなくてはならない。そのため read_binary 関数で buf に代入している。ファイルをデフォルトではなくバイナリで読み取るのは、ifstream で ios::binary とすることで可能となる。その後ファイルを先頭から読み取るイテレーターとファイルが終端に来たら読み取りを終了するイテレーターを用いてファイル全体を読み取り、それをすべて buf にコピーする。これを malware_search 関数に渡すことで、SIGNATURE を参照してマルウェア A ファミリーがあるかどうかを検索している。なお検索は全探索で行なっている。SIGNATURE のサイズを定義するマクロの SIG_LEN を用いているが、この大きさよりもそもそもファイルの大きさが小さいことも考えられるので、最初に例外処理を行なっている。これによりマルウェアが見つかったら、該当するファイル名を出力し、is_find フラグを立てる。そしてそのフラグをみて最終的な結果を出力することとなる。これがプログラムの概要である。

ファイル名のソートのために natural_num_comp 関数というものを使用しているが、デフォルトではファイルの検索はファイル名に則って行われないために出力がごちゃごちゃになる、というのを防ぐためにこの工夫を施した。ソート用の sort 関数に filename にあるファイル名をすべて渡し、自然順ソートの natural_num_comp 関数を渡して実行させている。ファイル名のソートだけであれば文字を一つずつ比較するだけで済むが、その場合、今回のファイルを例とすると、file1→file10→file2→…というように並んでしまう。これを防ぐために、ファイル名に数値が含まれていた場合は、それを計算して合算して合計を比較するという処理を施している。これによってファイルが綺麗に並ぶようになっている。

・実行結果

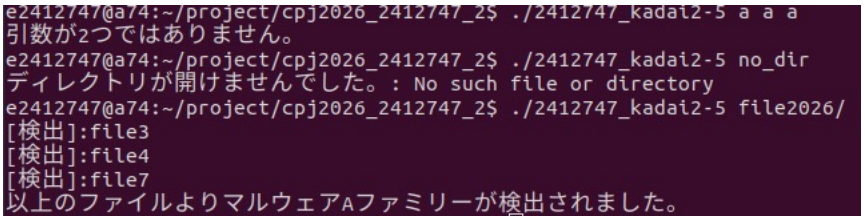

図5　問題5の実行結果